

Striver SDE Sheet — Arrays Part IV

1. Two Sum

Problem

Given an array and a target, find two different indices such that:

```
arr[i] + arr[j] = target
```

Example:

```
arr = [2,7,11,15]
target = 9

Answer = [0,1]
```

Approach 1: Brute Force

Check every pair.

```
for i from 0 to n-1:
    for j from i+1 to n-1:
        if arr[i] + arr[j] == target:
            return [i,j]
```

Complexity

- Time: $O(n^2)$
- Space: $O(1)$

Approach 2: Better — Hash Map

Idea

For every element:

```
required = target - arr[i]
```

Check whether `required` was previously seen.

```
map = value → index

for i from 0 to n-1:
    required = target - arr[i]

    if required exists in map:
        return [map[required], i]

    map[arr[i]] = i
```

Important

Check for the complement **before inserting** the current value. This prevents using the same element twice.

Complexity

- Time: `O(n)` average
 - Space: `O(n)`
-

Approach 3: Two Pointers

Use this when only the existence or values of the pair are needed.

Algorithm

```
sort(arr)

left = 0
right = n - 1

while left < right:
    sum = arr[left] + arr[right]

    if sum == target:
        return true

    else if sum < target:
        left++
```

```
else:
    right--
```

Complexity

- Time: $O(n \log n)$
- Space: $O(1)$ excluding sorting space

Limitation

Sorting changes the original indices. Store $\{\text{value}, \text{index}\}$ pairs if indices are required.

Revision key

For each value, search for $\text{target} - \text{value}$.

2. Four Sum

Problem

Find all unique quadruplets whose sum equals the target.

Example:

```
arr = [1,0,-1,0,-2,2]
target = 0
```

Answer:

```
[-2,-1,1,2]
[-2,0,0,2]
[-1,0,0,1]
```

Approach 1: Brute Force

Use four loops and check every quadruplet.

```
for i:
    for j:
```

```
    for k:
        for l:
            if arr[i] + arr[j] + arr[k] + arr[l] == target:
                add sorted quadruplet to set
```

A set removes duplicate quadruplets.

Complexity

- Time: $O(n^4)$
- Space: depends on the set

Approach 2: Better — Three Loops and Hash Set

Fix three indices:

```
i, j, k
```

Required fourth value:

```
fourth = target - arr[i] - arr[j] - arr[k]
```

Check whether `fourth` exists in a set of values seen during the third loop.

Complexity

- Time: $O(n^3)$
- Space: $O(n)$ plus result storage

Approach 3: Optimal — Sort and Two Pointers

Steps

1. Sort the array.
2. Fix the first two elements using `i` and `j`.
3. Find the other two elements using `left` and `right`.

```
sort(arr)

for i from 0 to n-1:
```

```
if i > 0 and arr[i] == arr[i-1]:
    continue

for j from i+1 to n-1:

    if j > i+1 and arr[j] == arr[j-1]:
        continue

    left = j + 1
    right = n - 1

    while left < right:

        sum = arr[i] + arr[j] + arr[left] + arr[right]

        if sum == target:
            add quadruplet
            left++
            right--

            skip duplicate values at left
            skip duplicate values at right

        else if sum < target:
            left++

        else:
            right--
```

Duplicate handling

Skip duplicate values for:

```
i
j
left
right
```

This ensures that every quadruplet is unique.

Overflow warning

Use a 64-bit integer:

```
long long sum
```

because the sum of four integers may overflow.

Complexity

- Sorting: $O(n \log n)$
- Main algorithm: $O(n^3)$
- Extra space: $O(1)$ excluding output and sorting

Revision key

Sort → fix two values → solve the remaining Two Sum using two pointers.

3. Longest Consecutive Sequence

Problem

Find the length of the longest sequence of consecutive integers.

The elements do not need to be adjacent in the original array.

Example:

```
arr = [100,4,200,1,3,2]
```

Longest sequence:

```
1,2,3,4
```

Answer:

```
4
```

Approach 1: Brute Force

For every element `x`, repeatedly search the array for:

`x+1, x+2, x+3...`

Complexity

- Time: approximately $O(n^2)$
 - Space: $O(1)$
-

Approach 2: Better — Sorting

Sort the array and count adjacent consecutive values.

```
sort(arr)

longest = 1
current = 1

for i from 1 to n-1:

    if arr[i] == arr[i-1]:
        continue

    else if arr[i] == arr[i-1] + 1:
        current++

    else:
        current = 1

    longest = max(longest, current)
```

Complexity

- Time: $O(n \log n)$
 - Space: depends on sorting
-

Approach 3: Optimal — Hash Set

Core idea

Start counting only from the first element of a sequence.

An element `x` is the beginning when:

`x - 1`

does not exist in the set.

Algorithm

```
insert all elements into a set

longest = 0

for x in set:
    if x - 1 does not exist:
        current = x
        length = 1

        while current + 1 exists:
            current++
            length++

        longest = max(longest, length)
```

Why check `x - 1`?

Consider:

1,2,3,4

Only `1` starts the sequence because `0` does not exist.

We avoid recounting the sequence from `2`, `3`, and `4`.

Complexity

- Time: $O(n)$ average
- Space: $O(n)$

Edge case

For an empty array:

```
return 0
```

Revision key

Begin a sequence only when its predecessor does not exist.

4. Largest Subarray with Sum Zero

Problem

Find the maximum length of a contiguous subarray whose sum is 0 .

Example:

```
arr = [9, -3, 3, -1, 6, -5]
```

Subarray:

```
[-3, 3, -1, 6, -5]
```

Sum:

```
0
```

Length:

```
5
```

Approach 1: Brute Force

Generate every subarray and calculate its sum.

```
maximumLength = 0

for i from 0 to n-1:
    for j from i to n-1:

        sum = 0

        for k from i to j:
            sum += arr[k]

        if sum == 0:
            maximumLength = max(maximumLength, j-i+1)
```

Complexity

- Time: $O(n^3)$
 - Space: $O(1)$
-

Approach 2: Better

Remove the third loop by maintaining a running sum.

```
for i from 0 to n-1:

    sum = 0

    for j from i to n-1:
        sum += arr[j]

    if sum == 0:
        maximumLength = max(maximumLength, j-i+1)
```

Complexity

- Time: $O(n^2)$
 - Space: $O(1)$
-

Approach 3: Optimal — Prefix Sum and Hash Map

Core observation

If the same prefix sum occurs at two indices:

```
prefix[i] == prefix[j]
```

then the sum between them is zero:

```
prefix[j] - prefix[i] = 0
```

Therefore, the subarray:

```
i+1 ... j
```

has sum zero.

Algorithm

Store:

```
prefixSum → first index where it appeared
```

```
sum = 0
maxLength = 0

for i from 0 to n-1:
    sum += arr[i]

    if sum == 0:
        maxLength = i + 1

    else if sum exists in map:
        length = i - map[sum]
        maxLength = max(maxLength, length)
```

```
else:  
    map[sum] = i
```

Why store the first occurrence only?

The earliest occurrence gives the longest possible subarray.

Do not update an existing prefix-sum index.

Complexity

- Time: $O(n)$ average
- Space: $O(n)$

Revision key

Repeated prefix sum means the elements between those positions sum to zero.

5. Count Subarrays with Given XOR

Problem

Count contiguous subarrays whose XOR equals K .

Example:

```
arr = [4,2,2,6,4]  
K = 6
```

Answer:

4

Approach 1: Brute Force

Generate every subarray and calculate its XOR separately.

```

for i:
    for j:
        xorValue = 0

        for k from i to j:
            xorValue ^= arr[k]

        if xorValue == K:
            count++

```

Complexity

- Time: $O(n^3)$
- Space: $O(1)$

Approach 2: Better

Maintain XOR while extending each subarray.

```

count = 0

for i from 0 to n-1:

    xorValue = 0

    for j from i to n-1:
        xorValue ^= arr[j]

    if xorValue == K:
        count++

```

Complexity

- Time: $O(n^2)$
- Space: $O(1)$

Approach 3: Optimal — Prefix XOR and Hash Map

XOR property

Let:

```
prefixXor = XOR from index 0 to i
```

Suppose the required subarray has XOR `K`.

```
previousPrefix XOR K = currentPrefix
```

Therefore:

```
previousPrefix = currentPrefix XOR K
```

For every current prefix XOR `xr`, search for:

```
xr XOR K
```

Algorithm

Store the frequency of every prefix XOR.

```
frequency[0] = 1

xr = 0
count = 0

for value in array:

    xr ^= value

    required = xr XOR K

    if required exists:
        count += frequency[required]

    frequency[xr]++
```

Why initialize `frequency[0] = 1`?

It handles subarrays beginning at index `0`.

Suppose:

```
prefixXor == K
```

Then:

```
required = prefixXor XOR K = 0
```

The initial zero prefix must be counted.

Complexity

- Time: $O(n)$ average
- Space: $O(n)$

Revision key

For current prefix XOR xr , search for the previous prefix $xr \oplus K$.

6. Longest Substring Without Repeating Characters

Problem

Find the length of the longest substring containing no repeated character.

Example:

```
s = "abcabcbb"
```

Longest substring:

```
"abc"
```

Answer:

```
3
```

Approach 1: Brute Force

Generate every substring and check whether all characters are unique.

```
for i from 0 to n-1:
    for j from i to n-1:
        check uniqueness of s[i...j]
```

Complexity

- Time: $O(n^3)$
- Space: $O(\text{character set size})$

Approach 2: Better — Sliding Window with Set

Maintain a window:

```
[left ... right]
```

containing unique characters.

When `s[right]` is already present:

- Remove characters from the left.
- Continue until the duplicate is removed.

```
left = 0
maxLength = 0
set = empty

for right from 0 to n-1:

    while s[right] exists in set:
        remove s[left] from set
        left++

    insert s[right]
    maxLength = max(maxLength, right-left+1)
```


Complexity

- Time: $O(2n) \rightarrow O(n)$
- Space: $O(\text{character set size})$

Each character is inserted and removed at most once.

Approach 3: Optimal — Store Last Seen Index

Instead of moving `left` one step at a time, jump directly after the previous occurrence.

Store:

```
character → latest index
```

Algorithm

```
left = 0
maxLength = 0

for right from 0 to n-1:

    char = s[right]

    if char was previously seen:
        left = max(left, lastSeen[char] + 1)

    lastSeen[char] = right

    maxLength =
        max(maxLength, right-left+1)
```

Why use `max(left, lastSeen[char] + 1)`?

The previous occurrence may be outside the current window.

Example:

```
"abba"
```

When processing the last `a`, its previous occurrence is at index `0`, but `left` is already at index `2`.

Without `max`, `left` would incorrectly move backward.

Complexity

- Time: `O(n)`
- Space: `O(character set size)`

Revision key

When a duplicate appears, move `left` directly after its last occurrence—but never backward.

Arrays Part IV — Final Revision Table

Problem	Optimal technique	Time	Extra space
Two Sum	Hash map	<code>O(n)</code> average	<code>O(n)</code>
Four Sum	Sorting + two pointers	<code>O(n³)</code>	<code>O(1)</code> *
Longest Consecutive Sequence	Hash set	<code>O(n)</code> average	<code>O(n)</code>
Largest Zero-Sum Subarray	Prefix sum + map	<code>O(n)</code> average	<code>O(n)</code>
Count Subarrays with XOR K	Prefix XOR + frequency map	<code>O(n)</code> average	<code>O(n)</code>
Longest Unique Substring	Sliding window + last index	<code>O(n)</code>	<code>O(charset)</code>

*Excluding output and sorting-related space.

One-Line Memory Tricks

Two Sum:
Look for `target - current value`.

Four Sum:
Sort, fix two values and use two pointers for the other two.

Longest Consecutive:
Start only when `value - 1` does not exist.

Zero-Sum Subarray:

Same prefix sum at two indices means zero sum between them.

Subarray XOR K:

Required previous XOR = current XOR K.

Longest Unique Substring:

Jump left after the duplicate's last position.

Important Prefix-Sum and XOR Formulas

Zero-Sum Subarray

```
prefix[j] - prefix[i] = 0
```

Therefore:

```
prefix[j] = prefix[i]
```

Subarray Sum Equal to K

```
requiredPrefix = currentPrefix - K
```

Subarray XOR Equal to K

```
requiredPrefixXor = currentPrefixXor XOR K
```

Pattern Recognition

Find two values producing a target

→ Hash map or two pointers

Find unique quadruplets

→ Sort + fix two + two pointers

Longest consecutive values in any order

→ Hash set and detect sequence starts

Longest zero-sum subarray
→ Prefix sum with earliest index

Count subarrays having XOR K
→ Prefix XOR frequency map

Longest substring with unique characters
→ Variable-size sliding window

Common Interview Mistakes

Two Sum

Do not use the same index twice.

Check the complement before inserting the current element.

Four Sum

Skip duplicates at all four pointer levels.
Use a 64-bit integer for the sum.

Longest Consecutive Sequence

Do not begin counting from every element.

Start only when $x - 1$ is absent.

Largest Zero-Sum Subarray

Store the first prefix-sum occurrence only.

Subarrays with XOR K

Initialize frequency[0] = 1.

Longest Unique Substring

Never move the left pointer backward.

Use:

```
left = max(left, lastSeen[character] + 1)
```